Legal Rights Navigator — Implementation Sandbox Playbook

Architecture Critique & Phase-by-Phase Execution Order

1. Architectural Critique

Before sequencing phases, four points in the original design need to be flagged. Two are foundational flaws worth fixing before any code is written; two are open questions the plan correctly identified but didn't resolve.

FLAW 1 — "REMOVING FROM CONVERSATION MEMORIES" IS NOT AN OPERATION

LangGraph state is just a Python dict that exists for one graph run. An invalid query already doesn't get added to `history_summary` unless you explicitly write it there. There is nothing to "remove." This needs to become a concrete rule in the state schema: *invalid queries are logged to a separate audit table for abuse monitoring, and are never written into extracted_facts or history_summary.* That's a one-line decision, not a feature to build.

FLAW 2 — DOMAIN CLASSIFICATION HAPPENS AFTER THE VALIDITY GATE, BUT ROUTING NEEDS IT EARLIER

The diagram validates legal/not-legal first, then classifies into 5 domains, then retrieves. But the retrieval step needs the domain to pick the right pgvector collection/filter. That's fine — the order is technically correct — but the planner step ("explain law / draft complaint / draft notice") cannot run a generic prompt across 5 domains with one shared prompt template. Each domain needs its own planner prompt and its own valid action set (e.g. Tenant domain doesn't have "file in Consumer Commission" as an option). This must be decided in Phase 2, not discovered in Phase 6.

CORRECTLY IDENTIFIED, NEEDS A DECISION NOW

"Document generator — what is this?" → This is template-filling, not generation. A complaint to a Labour Commissioner has a fixed legal structure with blanks (employer name, dates, wage amount). The LLM extracts facts into a structured JSON object; a Jinja2/docx template renders the final document. The LLM should never freeform-generate legal document text. This gets locked in at Phase 5.

CORRECTLY IDENTIFIED, NEEDS A DECISION NOW

"Who is the user?" → Already answered by the Postgres schema (Google OAuth sub as the permanent identifier). The schema in the doc is solid and should be implemented close to verbatim — no changes needed there.

2. Stack Decisions — Confirmed

FastAPI over Flask/Express, Postgres+pgvector over Qdrant/Milvus/Weaviate, and the proposed repo layout are all reasonable defaults for a single-team project at this scale and will not be revisited. We build on top of them as given.

3. Phase Order — Environment Setup to Production

Phase	Title	Goal	Est. Time
1.1	Environment & Repo Skeleton	Monorepo scaffold, Docker Compose (Postgres+pgvector, Redis), base FastAPI app boots	30–45 min
1.2	Database Models & Migrations	SQLAlchemy models for users/conversations/messages, first Alembic migration applied	45–60 min
1.3	Google OAuth + Session Cookies	Login flow working end-to-end, verified cookie issued and validated	45–60 min
2.1	Tier 1–2 Validity Gate	Regex blocklist + embedding similarity filter, no LLM call yet	30–45 min
2.2	Tier 3 LLM Classifier Gate + Domain Classifier	Haiku-tier binary gate for edge cases, then 5-domain classification with confidence score	45 min
3.1	Ingestion Pipeline — Chunking	Script: raw legal docs → chunked text, domain-tagged	45–60 min
3.2	Ingestion Pipeline — Embedding & Storage	Chunks embedded and stored in pgvector with domain metadata	30–45 min
3.3	Domain-Filtered Retriever + Reranker	Metadata pre-filter → semantic search → cross-encoder rerank	45–60 min
4.1	LangGraph State Schema	Formal TypedDict/Pydantic state object wiring all fields together	20–30 min
4.2	Graph Nodes — Validate / Clarify / Retrieve	First 3 nodes wired into a running graph with conditional edges	60 min
4.3	Graph Node — Legal Planner	Structured-output planner producing an Action Plan JSON per domain	45–60 min
5.1	Advice Generator	Structured JSON → conversational legal explanation with cited sources	30–45 min
5.2	Document/Complaint Generator (Template-Based)	Fact JSON → Jinja2/docx template fill, one domain's complaint form first	60 min
6.1	Conversation Memory & Summarization	Rolling summary + extracted_facts update logic per turn	45 min
6.2	FastAPI Chat Endpoint Wiring	Single `/chat` endpoint invoking the full graph, streaming response	45–60 min
7.1	Frontend Auth + Conversation List	Next.js login flow, conversation sidebar reading from backend	60 min
7.2	Frontend Chat Window	Message thread UI, checklist card rendering for structured responses	60–90 min
8.1	Remaining 4 Domains — Prompt & Checklist Pack	Replicate domain-specific prompt/checklist/template config for the other domains	90+ min
9.1	Testing & Eval Harness	Test set per domain for classifier accuracy + retrieval relevance	60 min
10.1	Production Hardening & Deploy	Rate limiting, logging, secrets, deployment target	90+ min

Note on sequencing: Phases 2 and 3 can technically run in parallel since the validity gate doesn't depend on retrieval, but they're presented serially here to avoid context-switching. Phase 8 (remaining domains) is deliberately placed late — Phases 1–7 are built end-to-end against a single domain (Labour & Employment) as the reference implementation, then replicated, rather than building all 5 domains shallowly at once.

Awaiting confirmation to serve Phase 1.1.